pds

Overview

`pds` is a library written in *Python* for computing and showing *proportionally dense subgraphs (PDSs)* of maximum possible size in any given graph. Alternatively, the *pds* library also allows generating and displaying random graphs of specific graph classes, such as cubic graphs, *k*-regular bipartite graphs, caterpillars, and trees. Thus, one or all PDSs of maximum possible size can be drawn on randomly generated instances of the above-mentioned graph classes. This library can be imported as a module into any *Python* project by using `import pds`.

Bazgan et al. defined *"a proportionally dense subgraph (PDS) as an induced subgraph of a graph with the property that each vertex in the PDS is adjacent to proportionally as many vertices in the subgraph as in the graph"* (source: https://arxiv.org/abs/1903.06579).

The project also contains a non-exhaustive list of *use cases* that show possible usage of the library. In addition, some use cases were used to test conjectures about PDSs in cubic graphs and *k*-regular bipartite graphs. The use cases are described in the section *Usage and Details* (see below).

Installation

On Debian-based Linux

In a `bash` terminal, type:

```
sudo apt update
sudo apt install python3-pip
git clone https://github.com/d-m-3/pds.git
pip install -r requirements.txt
```

On MacOS

1. Check that Python 3 is installed. In a terminal, type:

```
python3 --version
```

2. Download `pip`. In a terminal, type:

```
curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
```

3. Install `pip`. In a terminal, type:

```
python3 get-pip.py
```

4. Install the modules `networkx` and `matplotlib` with `pip`. In a terminal, type:

```
pip install networkx
pip install matplotlib
```

5. Get the files of the project. Two options:
   a) With `git clone`. In a terminal, type: `git clone https://github.com/d-m-3/pds.git`
   b) Or click on *Code - Download ZIP* on the project's GitHub page.

On Windows

1. To install `pip`, follow the `pip` documentation here.
2. Install the modules `networkx` and `matplotlib` with `pip`. In a terminal, type:

```
pip install networkx
pip install matplotlib
```

3. Get the files of the project. Two options:
   a) With `git clone`. In a terminal, type: `git clone https://github.com/d-m-3/pds.git`
   b) Or click on *Code - Download ZIP* on the project's GitHub page.

Usage and Details

- `pds.py` is the main library for computing and showing PDSs of maximum possible size in any given graph. Alternatively, the *pds* library also allows generating and displaying random graphs of specific graph classes, such as cubic graphs, *k*-regular bipartite graphs, caterpillars, and trees. Thus, one or all PDSs of maximum possible size can be drawn on randomly generated instances of the above-mentioned graph classes. This library can be imported as a module into any *Python* project by using `import pds` (see *External Usage* below).
- `pds_tests.py` contains the unit tests for all the functions in `pds.py`, except for the functions that draw and/or save graphs.
- `usecase_draw_1_pds.py`
  Goal: Draw a cubic graph or a *k*-regular bipartite graph and show one PDS of maximum possible size.
  Execution and details: It creates a random cubic graph or *k*-regular bipartite graph on a given number of vertices. It finds a PDS of maximum possible size, draws the graph, and colors the vertices of the PDS in red. By default, a circular layout is used to place the vertices. As an option, the "bipartite" layout can be used for bipartite graphs.
- `usecase_draw_all_pds.py`
  Goal: Draw all the possible PDSs of maximum possible size for a given cubic graph or *k*-regular bipartite graph.
  Execution and details: It creates a random cubic graph or *k*-regular bipartite graph on a given number of vertices. It finds all the PDSs of maximum possible size and draws them on different

figures (vertices belonging to a PDS are colored in red). By default, a circular layout is used to place the vertices. As an option, the "bipartite" layout can be used for bipartite graphs.

- `usecase_exceptions_cubic.py`
  Goal: Search for cubic graphs *G = (V, E)* with */V/ > 8* that admit no PDS of maximum possible size.
  Execution and details: It creates random cubic graphs and tests if they admit a PDS of maximum possible size. If a graph with no PDS of maximum possible size is found, then it is drawn, saved as a .png figure and as an edge list that can be imported later, and the program's execution is stopped. The number of vertices in graphs and the number of created and tested graphs can be defined. If the boolean "only_nh" is set to "True", then only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).
- `usecase_exceptions_k_regular_bipartite.py`
  Goal: Search for *k*-regular bipartite graphs *G = (V, E)* with */V/ > 8* that admit no PDS of maximum possible size.
  Execution and details: It creates random *k*-regular bipartite graphs and tests if they admit a PDS of maximum possible size. If a graph with no PDS of maximum possible size is found, then it is drawn, saved as a .png figure and as an edge list that can be imported later, and the program's execution is stopped. The number of vertices in graphs and the number of created and tested graphs can be defined.
- `usecase_every_v_in_pds.py`
  Goal: Test the following conjecture computationally "Let *G = (V, E)* be a cubic graph with */V/ > 8*. Then, every vertex in *V* is part of at least one PDS of maximum possible size".
  Execution and details: It creates random cubic graphs and tests if, for every graph, every vertex is part of at least one PDS of maximum possible size. If the program finds a graph in which some vertices are not part of any PDS, then the graph is drawn, the vertices not belonging to any PDS are displayed, and the program's execution is stopped. The number of vertices in graphs and the number of created and tested graphs can be defined. If the boolean "only_nh" is set to "True", then only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).
- `usecase_every_v_ds2.py`
  Goal: Test the following conjecture computationally "Let *G = (V, E)* be a cubic graph with */V/ > 8*. Then, there always exists a subset *S subset of V* such that *G[S]* is a PDS of maximum possible size and *d_S(u) = 2* for each vertex *u in S*". We proved that this conjecture was false by finding counterexamples.
  Execution and details: It creates random cubic graphs and tests if, for every graph, there exists at least one PDS of maximum possible size such that *d_S(u) = 2* for each vertex *u* in *S*. If there is no such PDS for a graph, the graph and all the PDSs of maximum possible size are drawn, and the program's execution is stopped. The number of vertices in graphs and the number of created and tested graphs can be defined. If the boolean "only_nh" is set to "True", then only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).
- `usecase_every_v_ds2_ds3.py`
  Goal: Test the following conjecture computationally "Let *G = (V, E)* be a cubic graph with */V/ > 8*. Then, there always exists a subset *S subset of V* such that *G[S]* is a PDS of maximum possible size and *d_S(u) = 3* for at most *i* vertices *u in S* for *i > 1*".
  Execution and details: It creates random cubic graphs and tests if, for every graph, there exists at least one PDS of maximum possible size such that *d_S(u) = 3* for at most `ds3_nb` vertices *u in S*. If there is no such PDS for a graph, the graph and all the PDSs of maximum possible size are drawn, and the program's execution is stopped. The number of vertices in graphs and the number of created and

tested graphs can be defined. If the boolean "only_nh" is set to "True", then only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).

- `usecase_create_nham_graph.py`
  Goal: Generate, display, and save random non-Hamiltonian cubic graphs on a given number of vertices.
  Execution and details: It creates a random non-Hamiltonian cubic graph on a given number of vertices and saves it in the given folder with the number of vertices at the end of the filename. It also saves the figure in .png format in the same directory.
- `usecase_tree.py`
  Goal: Create a random tree of maximum degree 3, find and draw a PDS of maximum possible size.
- `usecase_caterpillar.py`
  Goal: Create a random caterpillar of maximum degree 3, find and draw a PDS of maximum possible size.
- `usecase_tree_without_max_pds.py`
  Goal: Return a tree that does not admit a PDS of maximum possible size.
- `usecase_binomial.py`
  Goal: Create an Erdős-Rényi graph of maximum degree 3, and try to find and draw a PDS of maximum possible size.
- `gex.py`
  Contains specific graphs used for unit tests and specific cubic graphs on eight vertices that do not have a PDS of maximum possible size.

External Usage

You can import the `pds` library into any *Python* project:

```
import pds
```